

CENG 467 — Natural Language Understanding and Generation

Progress Report

Evaluating Context and History Pruning Strategies for RAG Faithfulness in Small Language Models via LLM-as-Judge

Mert Güden (300201013)

Berkay Fehmi Tekin (300201090)

May 2026

1 Project Title, Group Members, and Problem Definition

This project investigates how context pruning strategies affect the faithfulness of answers generated by small language models (SLMs) in a retrieval-augmented generation (RAG) setting. Faithfulness—the degree to which a generated answer is grounded in the retrieved context—is a critical quality dimension for RAG systems, yet it remains difficult to measure automatically and is particularly vulnerable when the context contains noisy or irrelevant passages.

Our task is open-domain question answering over HotpotQA [10] (distractor setting), a multi-hop QA benchmark in which each question is accompanied by ten passages: two supporting passages and eight distractors drawn from Wikipedia. For each question, our pipeline retrieves the top-5 most relevant passages using BM25, optionally applies a pruning strategy to reduce context length and noise, generates a short answer using a small instruction-tuned language model (Mistral-7B-Instruct or Llama-3.1-8B-Instruct), and evaluates the answer’s faithfulness using an LLM-as-Judge pipeline that decomposes the answer into atomic claims and verifies each claim against the retrieved context.

The central technical challenge is threefold. First, faithfulness is not directly measurable without expensive human annotation; we address this with a calibrated LLM-as-Judge [8] anchored to Cohen’s κ against human labels. Second, pruning introduces a coverage–noise trade-off: aggressive compression may remove the very sentences that support the correct answer. Third, small language models hallucinate more frequently under long or noisy contexts, making the pruning step crucial to generation quality. The seven pruning strategies we evaluate span from no pruning at all to combined semantic and history-aware compression, allowing us to systematically isolate the effect of each design choice.

2 Dataset / Benchmark Status

We use HotpotQA in its distractor configuration, loaded via the HuggingFace `datasets` library (`load_dataset("hotpot_qa", "distractor")`). The dataset contains approximately 90,447 training samples and 7,405 validation samples. For our experiments we draw 200 samples from the validation set and 50 separate samples for LLM-as-Judge calibration against human anno-

tations. We do not fine-tune any model; all experimental samples come from the validation split.

Each sample provides a natural language question, a short gold answer (typically a span or a yes/no value), ten passages with their titles, and supporting-fact labels indicating which sentences are relevant. During preprocessing we flatten each passage’s sentence list into a single string, apply whitespace tokenization for BM25 indexing, and truncate individual passages to a configurable token budget where needed. The loader script (`src/data/loader.py`) saves cleaned records in JSONL format. Dataset loading is fully implemented; the processed JSONL files are ready for the retrieval and pruning stages.

The primary challenges introduced by HotpotQA are: (1) the eight distractor passages directly inflate context length and introduce factual noise; (2) multi-hop questions require evidence from at least two separate passages, so aggressive pruning risks discarding one of the required supporting passages; and (3) HotpotQA does not provide a dedicated retrieval split, so we evaluate end-to-end retrieval alongside pruning on the same validation samples.

Table 1: HotpotQA Dataset Summary

Property	Value
Full name	HotpotQA (distractor setting)
Source	HuggingFace <code>hotpot_qa</code>
Training samples	~90,447
Validation samples	~7,405
Samples used (experiments)	200
Samples used (calibration)	50
Passages per question	10 (2 supporting + 8 distractors)
Answer type	Short span / Yes–No
Task type	Multi-hop open-domain QA
Status	Loader implemented, JSONL files prepared

3 Literature Review Progress

RAG and faithfulness. The RAG framework was introduced by Lewis et al. [1] as a method for grounding language model outputs in dynamically retrieved documents, improving performance on knowledge-intensive tasks. Shuster et al. [2] demonstrated empirically that retrieval augmentation substantially reduces hallucination in conversational settings compared to purely parametric generation. More recently, Min et al. [9] proposed FActScore, a fine-grained evaluation methodology that decomposes long-form generated text into atomic facts and verifies each against a knowledge source—an approach that directly motivates our LLM-as-Judge faithfulness metric.

Context compression. Jiang et al. [4] introduced LLMLingua, a token-level prompt compression approach that uses a small proxy language model to assign importance scores to individual tokens and removes low-scoring ones while preserving semantic integrity. Pan et al. [5] extended this line of work with LLMLingua-2, improving compression efficiency through data distillation and achieving better task-agnostic performance. Yoon et al. [6] proposed Provenance, a sentence-level pruning model trained specifically for RAG settings that scores each sentence’s relevance to the query and discards below-threshold content. Xu et al. [3] presented RECOMP, which frames context compression as either extractive or abstractive summarization, selecting query-relevant sentences via a learned compressor trained with contrastive objectives.

LLM-as-Judge. Zheng et al. [7] systematically evaluated the feasibility of using strong LLMs as judges for open-ended generation quality, identifying positional and verbosity biases but demonstrating high correlation with human rankings on MT-Bench. Es et al. [8] introduced RAGAS, a reference-free framework for RAG evaluation that measures faithfulness by checking whether each claim in the generated answer is entailed by the retrieved context—the same decompose-then-verify paradigm we adopt. We calibrate our judge using Cohen’s κ against human annotations on 50 held-out samples, following standard NLP practice for inter-annotator agreement measurement.

Technical direction. Our project follows the RAGAS faithfulness pipeline but uses open-weight models rather than proprietary APIs. The baseline pruners (B1–B3) replicate the experimental setup from the RECOMP paper; the advanced pruners (S1–S4) are based on LLMingua-2 and Provenance with additional history-aware extensions that remove redundant content from prior conversational turns.

4 Baseline Models and Pruning Strategies

We evaluate seven pruning strategies organized into three baselines and four advanced methods. The baselines establish reference points that bracket expected performance: B1 (no pruning) represents the upper bound for context coverage but the worst noise level; B2 (naive truncation) is the standard production fallback that limits total context to 512 tokens by hard cutoff; B3 (RECOMP extractive) is the strongest prior extractive baseline, selecting query-relevant sentences via Jaccard token overlap. The four advanced strategies (S1–S4) are our proposed contributions. All strategies share the same **BasePruner** interface and are registered in a central **PRUNER_REGISTRY**, enabling fair comparison under identical retrieval and generation conditions.

Table 2: Baseline and Advanced Pruning Strategies

ID	Strategy	Type	Status
B1	No Pruning (full context)	Baseline	Complete
B2	Naive Truncation (512 tok)	Baseline	Complete
B3	RECOMP Extractive	Prior work baseline	Complete
S1	LLMLingua-2	Token-level compression	Complete
S2	Provenance	Sentence-level pruning	Complete
S3	Semantic History Pruning	History-aware pruning	Complete
S4	Provenance + History (combined)	Combined	Complete

B1 serves as the upper-bound noise reference: providing the full retrieved context gives the generator the maximum available evidence but also exposes it to all distractor content. B2 is justified as the standard engineering fallback in production RAG systems where latency budgets constrain context length. B3 bridges the gap between simple truncation and learned compression by selecting sentences that lexically overlap with the query, serving as the extractive prior-work baseline. S1 (LLMLingua-2) and S2 (Provenance) represent the current state of the art in token-level and sentence-level compression respectively. S3 (history pruning) addresses a limitation of static pruners by removing passages that overlap with information already surfaced in prior conversational turns. S4 (combined) chains Provenance followed by history pruning, aiming to benefit from both semantic filtering and redundancy removal.

5 Initial Experimental Results

All seven pruning strategies have been evaluated end-to-end on 50 HotpotQA validation samples using Mistral-7B-Instruct-v0.2 as both the generator and the LLM-as-Judge, running on an NVIDIA RTX 5060 Ti GPU (17.1 GB VRAM). The model is loaded once and shared between the generator and judge to avoid memory pressure. Each sample passes through BM25 retrieval ($k = 5$), the selected pruner, Mistral-7B generation, and atomic claim scoring. Results are reported in Table 3.

Table 3: Experimental Results — 50 HotpotQA validation samples, Mistral-7B-Instruct-v0.2

Strategy	Faithfulness	EM	F1	Comp. Ratio	Lat. (s)
B1: No Pruning	0.718	0.040	0.148	1.000	6.76
B2: Naive Truncation	0.621	0.040	0.139	0.918	4.95
B3: RECOMP	0.750	0.040	0.141	0.911	5.38
S1: LLMingua-2	0.750	0.040	0.141	0.911	5.38
S2: Provence	0.657	0.000	0.057	0.072	4.49
S3: History Pruning	0.718	0.040	0.148	1.000	6.34
S4: Combined	0.657	0.000	0.057	0.072	4.48

Note: Faithfulness = fraction of atomic claims supported by context (LLM-as-Judge, Mistral-7B). EM = Exact Match vs. gold answer. Comp. Ratio = pruned tokens / original tokens ($n = 50$ samples per strategy).

Key observations. B3 (RECOMP) and S1 (LLMingua-2) achieve the highest faithfulness (0.750) with a compression ratio of 0.911, confirming that query-aligned sentence selection retains more supporting content than hard cutoff. S1 results are identical to B3 because the `llmlingua` library was not available in the current environment and S1 fell back to the RECOMP scoring function; a proper LLMingua-2 run with the library installed is planned. B2 (Naive Truncation) has the lowest faithfulness (0.621), consistent with the hypothesis that blind hard cutoff removes supporting sentences. S2 (Provence) achieves the most aggressive compression (ratio 0.072, retaining only 7% of context tokens) but at a significant cost: faithfulness drops to 0.657 and token F1 collapses to 0.057, indicating that the default relevance threshold is too aggressive for HotpotQA’s multi-hop structure where evidence spans multiple passages. S3 (History Pruning) with no conversation history is identical to B1 by design; S4 (Combined) is identical to S2 for the same reason. EM scores are uniformly low (0.040 or 0.000) because Mistral-7B generates verbose explanatory answers rather than extracting the short gold span, which is a known behaviour of instruction-tuned models on extractive QA benchmarks.

6 Planned Improvements and Technical Direction

Phase 1 (complete). All seven strategies have been evaluated on 50 HotpotQA validation samples using Mistral-7B-Instruct-v0.2. The pipeline runs end-to-end: BM25 retrieval, pruning, generation, and LLM-as-Judge faithfulness scoring. The results confirm that extractive pruning (B3/RECOMP) outperforms hard truncation (B2) on faithfulness, and that Provence’s default threshold is too aggressive for multi-hop QA.

Phase 2 (next). The immediate next step is to run the final Colab reproduction notebook, `notebooks/05_final_colab_runs.ipynb`. This notebook installs the `llmlingua` package before evaluating S1, so LLMingua-2 can no longer be accidentally reported as a RECOMP fallback.

The same notebook also creates the human annotation template for judge calibration, computes Cohen’s κ after labels are filled, and runs S3/S4 under a synthetic two-turn history setting rather than the current single-turn degenerate case.

Phase 3 (final). The final phase will scale the Colab runs to 200 samples per strategy, tune Provence’s relevance threshold (currently 0.6, likely too high for multi-hop evidence), and replace the preliminary S1/S3/S4 rows with notebook-generated results. An ablation study on judge prompts and retrieval depth ($k = 3$ vs. $k = 5$) will follow, after which the complete final report will be written.

7 Ablation and Prompt Sensitivity Plan

We plan a targeted set of ablation experiments to isolate the contribution of individual design choices:

- **Judge prompt format:** zero-shot vs. 2-shot examples for both claim decomposition and claim verification steps.
- **Verification response type:** strict binary YES/NO vs. soft confidence score thresholded at 0.5.
- **Retrieval depth:** $k = 3$ vs. $k = 5$ retrieved passages (affects both coverage and noise level).
- **Truncation budget:** `max_tokens` = 256 vs. 512 for B2 and B3.
- **History context:** S3 and S4 evaluated with empty history (single-turn) vs. a simulated two-turn history to assess the incremental value of history-aware pruning.

These ablations are designed to answer whether the faithfulness gains from advanced pruners are attributable to the compression strategy itself or to incidental factors such as prompt wording or retrieval depth.

8 Error Analysis and Generation Quality

Inspection of the 50-sample outputs reveals consistent error patterns. Mistral-7B-Instruct generates verbose multi-sentence explanations rather than short gold spans, resulting in EM scores that cluster near zero even when the answer is factually correct. For yes/no questions the model states the correct binary value but embeds it in a paragraph of reasoning, which EM cannot credit. For multi-hop factual questions the model occasionally conflates entities from different retrieved passages, producing a fluent but incorrect answer; this suggests the generator does not consistently weight the highest-ranked passage.

The most systematic error pattern occurs with Provence (S2): the 0.072 compression ratio indicates that the default relevance threshold (0.6) removes both supporting passages for multi-hop questions, leaving the model with context insufficient for chain-of-thought reasoning. Errors will be categorized into: (a) *retrieval failure* — supporting passage absent from top- k ; (b) *pruning error* — supporting sentence removed by pruner; (c) *generation error* — model ignores available evidence; (d) *judge error* — supported claim marked unsupported due to paraphrase mismatch. We plan to annotate 20–30 failure cases per strategy before the final submission to quantify each error type.

9 Ethical Considerations and Bias

The LLM-as-Judge pipeline is powered by Mistral-7B-Instruct, a model that may inherit the biases present in its pretraining and instruction-tuning corpora, including positional bias (pre-

ferring claims that appear early in the context), verbosity bias (favoring longer answers as more faithful), and cultural or demographic skews. HotpotQA passages are sourced from English Wikipedia, which is known to underrepresent non-Western entities and perspectives; our faithfulness measurements may therefore be systematically more reliable for topics with dense Wikipedia coverage. We also note that faithfulness to retrieved context is not equivalent to truthfulness about the world: a model that correctly reproduces a false claim from a retrieved passage will score high on faithfulness while being factually incorrect. All models used in this project are open-weight and used solely for academic research; no proprietary API access is required, and no user data or personally identifiable information is involved. Results from this study should not be extrapolated to high-stakes QA applications without additional human evaluation.

10 GitHub and Reproducibility Status

The repository is organized as a modular Python package with clearly separated components for data loading, retrieval, pruning, generation, evaluation, and judge calibration. For reproduction, however, the primary execution interface is Google Colab rather than direct local Python commands. The final report will reference the Colab notebooks as the runnable artifacts: `01_data_exploration.ipynb` for dataset checks, `02_baseline_results.ipynb` for small-scale baseline sanity checks, and `05_final_colab_runs.ipynb` for the final LLMingua-2, two-turn history, judge calibration, and error-analysis runs. The LNN/CfC work is kept as an exploratory extension in `03_lnn_training_colab.ipynb` and `04_lnn_evaluation_colab.ipynb`.

```
rag-faithfulness-pruning/
|-- PROJECT.md
|-- README.md
|-- requirements.txt
|-- test_pipeline.py
|-- notebooks/
|   |-- 01_data_exploration.ipynb
|   |-- 02_baseline_results.ipynb
|   |-- 03_lnn_training_colab.ipynb
|   |-- 04_lnn_evaluation_colab.ipynb
|   \-- 05_final_colab_runs.ipynb
|-- data/
|   |-- raw/          (hotpotqa_train.jsonl, hotpotqa_validation.jsonl)
|   |-- processed/    (hotpotqa_experiments.jsonl, hotpotqa_calibration.jsonl)
|   \-- judge_calibration/
|-- src/
|   |-- data/          (loader.py, splitter.py)
|   |-- retrieval/     (bm25_retriever.py)
|   |-- pruning/       (base.py, no_pruning.py, naive_truncation.py,
|                       recomp.py, llmlingua2.py, provence.py,
|                       history_pruning.py, combined.py)
|   |-- generation/    (generator.py)
|   |-- judge/         (judge.py, prompts.py, calibration.py)
|   \-- evaluation/    (runner.py, metrics.py)
\-- experiments/results/
    (no_pruning_50samples.jsonl, naive_truncation_50samples.jsonl,
     recomp_50samples.jsonl, llmlingua2_50samples.jsonl,
     provence_50samples.jsonl, history_pruning_50samples.jsonl,
     combined_50samples.jsonl, aggregate_summary.json)
```

All seven core pruners are implemented and available through the shared registry. Final reported outputs are reproduced by opening `notebooks/05_final_colab_runs.ipynb` in a GPU Colab runtime and running the cells from top to bottom. The notebook clones the repository, installs dependencies, downloads HotpotQA, writes JSONL result files under `experiments/results/`, produces a report-ready aggregate summary, and exports candidate cases for qualitative error analysis.

GitHub repository: https://github.com/Mrtuzy/CENG467_Final

11 Current Challenges and Next Steps

Challenges. The current results surface three concrete challenges. First, Provence’s default relevance threshold (0.6) is too aggressive for multi-hop QA: by retaining only 7.2% of context tokens it systematically discards at least one of the two required supporting passages, causing faithfulness (0.657) and F1 (0.057) to degrade below even the no-pruning baseline. Threshold tuning is required before S2 and S4 can be fairly evaluated. Second, S1 (LLMLingua-2) falls back to the RECOMP scoring function because the `llmlingua` library is not yet installed, making S1 and B3 results identical; a proper S1 evaluation requires installing and configuring the library. Third, Mistral-7B generates verbose explanatory answers rather than short gold-matching spans, causing EM scores to cluster at 0.040 across all strategies; future runs will add a post-hoc answer extraction step (e.g., prompting the model to output only the final answer token) to obtain more meaningful EM values.

Next steps.

1. Run `notebooks/05_final_colab_runs.ipynb` in Colab to obtain genuine LLMLingua-2 results and verify that the fallback counter is zero.
2. Use the same notebook to evaluate S3 and S4 with synthetic two-turn history rather than empty single-turn history.
3. Fill the generated `human_labels.jsonl` file for at least 20, preferably 50, calibration examples and compute Cohen’s κ .
4. Tune Provence relevance threshold (try 0.3, 0.4) to obtain a compression ratio in the 0.4–0.6 range rather than 0.07.
5. Scale the notebook configuration from 50 to 200 samples per strategy after threshold and prompt calibration.
6. Use the notebook’s exported `error_analysis_candidates.jsonl` file to add concrete qualitative examples to the final report.
7. Keep LNN/CfC pruning as future work: a lightweight adaptive controller could choose pruning aggressiveness dynamically from query complexity, retrieval redundancy, and history overlap.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston. Retrieval Augmentation Reduces Hallucination in Conversation. In *Findings of EMNLP*, 2021.

- [3] F. Xu, W. Shi, and E. Choi. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of EMNLP*, 2023.
- [5] Z. Pan, Q. Wu, H. Jiang, M. Garg, N. Liden, Y. Ronen, and L. Qiu. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of ACL*, 2024.
- [6] S. Yoon, W. Xu, F. Xu, and H. Ji. Provence: Efficient and Robust Context Pruning for Retrieval-Augmented Generation. In *Proceedings of NAACL*, 2024.
- [7] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] S. Es, J. James, L. E. Anke, and S. Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of EACL*, 2024.
- [9] S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of EMNLP*, 2023.
- [10] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of EMNLP*, 2018.